Note that in this chapter, as elsewhere in the text, we use hyphenated names for schema, relations, and attributes for ease of reading. In actual SQL systems, however, hyphens are not valid parts of a name (they are treated as the minus operator). A simple way of translating the names we use to valid SQL names is to replace all hyphens by the underscore symbol ("_"). For example, we use *branch_name* in place of *branch-name*.

4.2 Basic Structure

A relational database consists of a collection of relations, each of which is assigned a unique name. Each relation has a structure similar to that presented in Chapter 3. SQL allows the use of null values to indicate that the value either is unknown or does not exist. It allows a user to specify which attributes cannot be assigned null values, as we shall discuss in Section 4.11.

The basic structure of an SQL expression consists of three clauses: **select**, **from**, and **where**.

- The **select** clause corresponds to the projection operation of the relational algebra. It is used to list the attributes desired in the result of a query.
- The **from** clause corresponds to the Cartesian-product operation of the relational algebra. It lists the relations to be scanned in the evaluation of the expression.
- The **where** clause corresponds to the selection predicate of the relational algebra. It consists of a predicate involving attributes of the relations that appear in the **from** clause.

That the term *select* has different meaning in SQL than in the relational algebra is an unfortunate historical fact. We emphasize the different interpretations here to minimize potential confusion.

A typical SQL query has the form

$$\begin{aligned}&\textbf{select } A_1,\ A_2, \ldots, A_n\\ &\textbf{from } r_1,\ r_2, \ldots, r_m\\ &\textbf{where } P\end{aligned}$$

Each A_i represents an attribute, and each r_i a relation. P is a predicate. The query is equivalent to the relational-algebra expression

$$\Pi_{A_1,\ A_2, \ldots, A_n}(\sigma_P\,(r_1\ \times\ r_2\ \times\ \cdots\ \times\ r_m))$$

If the **where** clause is omitted, the predicate P is **true**. However, unlike the result of a relational-algebra expression, the result of the SQL query may contain multiple copies of some tuples; we shall return to this issue in Section 4.2.8.

SQL forms the Cartesian product of the relations named in the **from** clause, performs a relational-algebra selection using the **where** clause predicate, and then

projects the result onto the attributes of the **select** clause. In practice, SQL may convert the expression into an equivalent form that can be processed more efficiently. However, we shall defer concerns about efficiency to Chapters 13 and 14.

4.2.1 The select Clause

The result of an SQL query is, of course, a relation. Let us consider a simple query using our banking example, "Find the names of all branches in the *loan* relation":

select *branch-name*
from *loan*

The result is a relation consisting of a single attribute with the heading *branch-name*.

Formal query languages are based on the mathematical notion of a relation being a set. Thus, duplicate tuples never appear in relations. In practice, duplicate elimination is time-consuming. Therefore, SQL (like most other commercial query languages) allows duplicates in relations as well as in the results of SQL expressions. Thus, the preceding query will list each *branch-name* once for every tuple in which it appears in the *loan* relation.

In those cases where we want to force the elimination of duplicates, we insert the keyword **distinct** after **select**. We can rewrite the preceding query as

select distinct *branch-name*
from *loan*

if we want duplicates removed.

SQL allows us to use the keyword **all** to specify explicitly that duplicates are not removed:

select all *branch-name*
from *loan*

Since duplicate retention is the default, we will not use **all** in our examples. To ensure the elimination of duplicates in the results of our example queries, we will use **distinct** whenever it is necessary. In most queries where **distinct** is not used, the exact number of duplicate copies of each tuple present in the query result is not important. However, the number is important in certain applications; we return to this issue in Section 4.2.8.

The asterisk symbol " * " can be used to denote "all attributes." Thus, the use of *loan*.* in the preceding **select** clause would indicate that all attributes of *loan* are to be selected. A select clause of the form **select** * indicates that all attributes of all relations appearing in the **from** clause are selected.

The **select** clause may also contain arithmetic expressions involving the operators $+$, $-$, $*$, and / operating on constants or attributes of tuples. For example, the query

select *loan-number*, *branch-name*, *amount* * 100
from *loan*

will return a relation that is the same as the *loan* relation, except that the attribute *amount* is multiplied by 100.

SQL also provides special data types, such as various forms of the *date* type, and allows several arithmetic functions to operate on these types.

4.2.2 The where Clause

Let us illustrate the use of the **where** clause in SQL. Consider the query "Find all loan numbers for loans made at the Perryridge branch with loan amounts greater that $1200." This query can be written in SQL as:

select *loan-number*
from *loan*
where *branch-name* = 'Perryridge' **and** *amount* > 1200

SQL uses the logical connectives **and**, **or**, and **not**—rather than the mathematical symbols $\wedge$, $\vee$, and $\neg$ —in the **where** clause. The operands of the logical connectives can be expressions involving the comparison operators <, <=, >, >=, =, and <>. SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types.

SQL includes a **between** comparison operator to simplify **where** clauses that specify that a value be less than or equal to some value and greater than or equal to some other value. If we wish to find the loan number of those loans with loan amounts between $90,000 and $100,000, we can use the **between** comparison to write

select *loan-number*
from *loan*
where *amount* **between** 90000 **and** 100000

instead of

select *loan-number*
from *loan*
where *amount* <= 100000 **and** *amount* >= 90000

Similarly, we can use the **not between** comparison operator.

4.2.3 The from Clause

Finally, let us discuss the use of the **from** clause. The **from** clause by itself defines a Cartesian product of the relations in the clause. Since the natural join is defined in terms of a Cartesian product, a selection, and a projection, it is a relatively simple matter to write an SQL expression for the natural join.

We write the relational-algebra expression

$$\Pi_{customer\text{-}name,\, loan\text{-}number,\, amount}\ (borrower \bowtie loan)$$

for the query "For all customers who have a loan from the bank, find their names, loan numbers and loan amount." In SQL, this query can be written as

select *customer-name, borrower.loan-number, amount*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number*

Notice that SQL uses the notation *relation-name.attribute-name*, as does the relational algebra, to avoid ambiguity in cases where an attribute appears in the schema of more than one relation. We could have written *borrower.customer-name* instead of *customer-name* in the **select** clause. However, since the attribute *customer-name* appears in only one of the relations named in the **from** clause, there is no ambiguity when we write *customer-name*.

We can extend the preceding query and consider a more complicated case in which we require also that the loan be from the Perryridge branch: "Find the customer names, loan numbers, and loan amounts for all loans at the Perryridge branch." To write this query, we need to state two constraints in the **where** clause, connected by the logical connective **and**:

select *customer-name, borrower.loan-number, amount*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number* **and**
branch-name = 'Perryridge'

SQL includes extensions to perform natural joins and outer joins in the **from** clause. We discuss these extensions in Section 4.10.

4.2.4 The Rename Operation

SQL provides a mechanism for renaming both relations and attributes. It uses the **as** clause, taking the form:

old-name **as** *new-name*

The **as** clause can appear in both the **select** and **from** clauses.

Consider again the query that we used earlier:

select *customer-name, borrower.loan-number, amount*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number*

The result of this query is a relation with the following attributes:

customer-name, loan-number, amount.

The names of the attributes in the result are derived from the names of the attributes in the relations in the **from** clause.

We cannot, however, always derive names in this way, for several reasons: First, two relations in the **from** clause may have attributes with the same name, in which case an attribute name is duplicated in the result. Second, if we used an arithmetic expression in the **select** clause, the resultant attribute does not have a name. Third,

even if an attribute name can be derived from the base relations as in the preceding example, we may want to change the attribute name in the result. Hence, SQL provides a way of renaming the attributes of a result relation.

For example, if we want the attribute name *loan-number* to be replaced with the name *loan-id*, we can rewrite the preceding query as

select *customer-name, borrower.loan-number* **as** *loan-id, amount*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number*

4.2.5 Tuple Variables

The **as** clause is particularly useful in defining the notion of tuple variables, as is done in the tuple relational calculus. A tuple variable in SQL must be associated with a particular relation. Tuple variables are defined in the **from** clause by way of the **as** clause. To illustrate, we rewrite the query "For all customers who have a loan from the bank, find their names, loan numbers, and loan amount" as

select *customer-name, T.loan-number, S.amount*
from *borrower* **as** *T*, *loan* **as** *S*
where *T.loan-number* = *S.loan-number*

Note that we define a tuple variable in the **from** clause by placing it after the name of the relation with which it is associated, with the keyword **as** in between (the keyword **as** is optional). When we write expressions of the form *relation-name.attribute-name*, the relation name is, in effect, an implicitly defined tuple variable.

Tuple variables are most useful for comparing two tuples in the same relation. Recall that, in such cases, we could use the rename operation in the relational algebra. Suppose that we want the query "Find the names of all branches that have assets greater than at least one branch located in Brooklyn." We can write the SQL expression

select distinct *T.branch-name*
from *branch* **as** *T*, *branch* **as** *S*
where *T.assets* > *S.assets* **and** *S.branch-city* = 'Brooklyn'

Observe that we could not use the notation *branch.asset*, since it would not be clear which reference to *branch* is intended.

SQL permits us to use the notation $(v_1, v_2, \ldots, v_n)$ to denote a tuple of arity n containing values $v_1, v_2, \ldots, v_n$. The comparison operators can be used on tuples, and the ordering is defined lexicographically. For example, $(a_1, a_2) <= (b_1, b_2)$ is true if $a_1 < b_1$, or $(a_1 = b_1) \wedge (a_2 <= b_2)$; similarly, the two tuples are equal if all their attributes are equal.

4.2.6 String Operations

SQL specifies strings by enclosing them in single quotes, for example, 'Perryridge', as we saw earlier. A single quote character that is part of a string can be specified by

using two single quote characters; for example the string "It's right" can be specified by 'It''s right'.

The most commonly used operation on strings is pattern matching using the operator **like**. We describe patterns by using two special characters:

- Percent (%): The % character matches any substring.
- Underscore (_): The _ character matches any character.

Patterns are case sensitive; that is, uppercase characters do not match lowercase characters, or vice versa. To illustrate pattern matching, we consider the following examples:

- 'Perry%' matches any string beginning with "Perry".
- '%idge%' matches any string containing "idge" as a substring, for example, 'Perryridge', 'Rock Ridge', 'Mianus Bridge', and 'Ridgeway'.
- '_ _ _' matches any string of exactly three characters.
- '_ _ _%' matches any string of at least three characters.

SQL expresses patterns by using the **like** comparison operator. Consider the query "Find the names of all customers whose street address includes the substring 'Main'." This query can be written as

select *customer-name*
from *customer*
where *customer-street* **like** '%Main%'

For patterns to include the special pattern characters (that is, % and _), SQL allows the specification of an escape character. The escape character is used immediately before a special pattern character to indicate that the special pattern character is to be treated like a normal character. We define the escape character for a **like** comparison using the **escape** keyword. To illustrate, consider the following patterns, which use a backslash (\) as the escape character:

- **like** 'ab\%cd%' **escape** '\' matches all strings beginning with "ab%cd".
- **like** 'ab\\cd%' **escape** '\' matches all strings beginning with "ab\cd".

SQL allows us to search for mismatches instead of matches by using the **not like** comparison operator.

SQL also permits a variety of functions on character strings, such as concatenating (using "||"), extracting substrings, finding the length of strings, converting between uppercase and lowercase, and so on. SQL:1999 also offers a **similar to** operation, which provides more powerful pattern matching than the **like** operation; the syntax for specifying patterns is similar to that used in Unix regular expressions.

4.2.7 Ordering the Display of Tuples

SQL offers the user some control over the order in which tuples in a relation are displayed. The **order by** clause causes the tuples in the result of a query to appear in sorted order. To list in alphabetic order all customers who have a loan at the Perryridge branch, we write

select distinct *customer-name*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number* **and**
branch-name = 'Perryridge'
order by *customer-name*

By default, the **order by** clause lists items in ascending order. To specify the sort order, we may specify **desc** for descending order or **asc** for ascending order. Furthermore, ordering can be performed on multiple attributes. Suppose that we wish to list the entire *loan* relation in descending order of *amount*. If several loans have the same amount, we order them in ascending order by loan number. We express this query in SQL as follows:

select *
from *loan*
order by *amount* **desc**, *loan-number* **asc**

To fulfill an **order by** request, SQL must perform a sort. Since sorting a large number of tuples may be costly, it should be done only when necessary.

4.2.8 Duplicates

Using relations with duplicates offers advantages in several situations. Accordingly, SQL formally defines not only what tuples are in the result of a query, but also how many copies of each of those tuples appear in the result. We can define the duplicate semantics of an SQL query using *multiset* versions of the relational operators. Here, we define the multiset versions of several of the relational-algebra operators. Given multiset relations r_1 and r_2,

1. If there are c_1 copies of tuple t_1 in r_1, and t_1 satisfies selection σ_θ, then there are c_1 copies of t_1 in $\sigma_\theta(r_1)$.
2. For each copy of tuple t_1 in r_1, there is a copy of tuple $\Pi_A(t_1)$ in $\Pi_A(r_1)$, where $\Pi_A(t_1)$ denotes the projection of the single tuple t_1.
3. If there are c_1 copies of tuple t_1 in r_1 and c_2 copies of tuple t_2 in r_2, there are $c_1 * c_2$ copies of the tuple $t_1.t_2$ in $r_1 \times r_2$.

For example, suppose that relations r_1 with schema (A, B) and r_2 with schema (C) are the following multisets:

$$r_1 = \{(1, a), (2, a)\} \qquad r_2 = \{(2), (3), (3)\}$$